

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

a = np.exp(1j * 2 * np.pi / 3)
A_MATRIX = np.array([[1, 1, 1],
                     [1, a**2, a],
                     [1, a, a**2]])

FAULT_TYPES = ["No Fault", "LG", "LL", "LLG", "LLL"]
```

```
def sequence_currents(fault_type, Z1, Z2, Z0, Zf, E=1.0):

    if fault_type == "No Fault":
        Ia1 = E / (Z1 * 8)
        return 0j, Ia1, 0j

    if fault_type == "LLL":
        Ia1 = E / (Z1 + Zf)
        return 0j, Ia1, 0j

    if fault_type == "LG":
        Ia1 = E / (Z1 + Z2 + Z0 + 3 * Zf)
        return Ia1, Ia1, Ia1

    if fault_type == "LL":
        Ia1 = E / (Z1 + Z2 + Zf)
        return 0j, Ia1, -Ia1

    if fault_type == "LLG":
        Za = Z2 * (Z0 + 3 * Zf) / (Z2 + Z0 + 3 * Zf)
        Ia1 = E / (Z1 + Za)
        Ia2 = -Ia1 * (Z0 + 3 * Zf) / (Z2 + Z0 + 3 * Zf)
        Ia0 = -Ia1 * Z2 / (Z2 + Z0 + 3 * Zf)
        return Ia0, Ia1, Ia2

    raise ValueError(fault_type)

def phase_quantities(Ia0, Ia1, Ia2, Z1, Z2, Z0, E=1.0):
    Va1 = E - Ia1 * Z1
    Va2 = -Ia2 * Z2
    Va0 = -Ia0 * Z0

    I_seq = np.array([Ia0, Ia1, Ia2])
    V_seq = np.array([Va0, Va1, Va2])

    I_phase = A_MATRIX @ I_seq
    V_phase = A_MATRIX @ V_seq
    return V_phase, I_phase
```

```
N_PER_CLASS = 2000
records = []

for fault_type in FAULT_TYPES:
    for _ in range(N_PER_CLASS):
        R1 = np.random.uniform(0.02, 0.08)
        X1 = np.random.uniform(0.15, 0.35)
        Z1 = complex(R1, X1)
        Z2 = Z1
        Z0 = complex(R1 * np.random.uniform(1.5, 3.0), X1 * np.random.uniform(2.0, 4.0))
        Rf = np.random.uniform(0, 15) if fault_type != "No Fault" else 0.0
        Zf = complex(Rf, 0)

        Ia0, Ia1, Ia2 = sequence_currents(fault_type, Z1, Z2, Z0, Zf)
        V_phase, I_phase = phase_quantities(Ia0, Ia1, Ia2, Z1, Z2, Z0)

        shift = np.random.choice([0, 1, 2])
        V_phase = np.roll(V_phase, shift)
        I_phase = np.roll(I_phase, shift)
```

```

V_noise = np.random.normal(0, 0.01, size=3)
I_noise = np.random.normal(0, 0.02, size=3)
V_phase = V_phase + V_noise
I_phase = I_phase + I_noise

record = {
    "Va": abs(V_phase[0]), "Vb": abs(V_phase[1]), "Vc": abs(V_phase[2]),
    "Ia": abs(I_phase[0]), "Ib": abs(I_phase[1]), "Ic": abs(I_phase[2]),
    "I0_mag": abs(Ia0), "I1_mag": abs(Ia1), "I2_mag": abs(Ia2),
    "fault_type": fault_type,
}
records.append(record)

df = pd.DataFrame(records)
df["V_unbalance"] = df[["Va", "Vb", "Vc"]].max(axis=1) - df[["Va", "Vb", "Vc"]].min(axis=1)
df["I_unbalance"] = df[["Ia", "Ib", "Ic"]].max(axis=1) - df[["Ia", "Ib", "Ic"]].min(axis=1)

print(f"Generated {len(df)} samples across {len(FAULT_TYPES)} fault classes")
print(df["fault_type"].value_counts())
df.head()

```

Generated 10000 samples across 5 fault classes

fault\_type

No Fault 2000

LG 2000

LL 2000

LLG 2000

LLL 2000

Name: count, dtype: int64

|   | Va       | Vb       | Vc       | Ia       | Ib       | Ic       | I0_mag | I1_mag   | I2_mag | fault_type | V_unbalance | I_unbalance |
|---|----------|----------|----------|----------|----------|----------|--------|----------|--------|------------|-------------|-------------|
| 0 | 0.879620 | 0.875621 | 0.854890 | 0.373761 | 0.370952 | 0.362827 | 0.0    | 0.364661 | 0.0    | No Fault   | 0.024730    | 0.010934    |
| 1 | 0.889624 | 0.867409 | 0.887447 | 0.424386 | 0.427798 | 0.420564 | 0.0    | 0.423260 | 0.0    | No Fault   | 0.022215    | 0.007234    |
| 2 | 0.863490 | 0.873128 | 0.878019 | 0.533753 | 0.546735 | 0.562322 | 0.0    | 0.535149 | 0.0    | No Fault   | 0.014529    | 0.028569    |
| 3 | 0.881394 | 0.885920 | 0.861445 | 0.518385 | 0.499439 | 0.482852 | 0.0    | 0.498943 | 0.0    | No Fault   | 0.024475    | 0.035533    |
| 4 | 0.860215 | 0.878621 | 0.877312 | 0.688895 | 0.673127 | 0.659949 | 0.0    | 0.679963 | 0.0    | No Fault   | 0.018407    | 0.028946    |

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score

```

```

FEATURES = ["Va", "Vb", "Vc", "Ia", "Ib", "Ic",
            "I0_mag", "I1_mag", "I2_mag", "V_unbalance", "I_unbalance"]

```

```

le = LabelEncoder()
y = le.fit_transform(df["fault_type"])
X = df[FEATURES].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

```

```

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f"Train: {len(y_train)}, Test: {len(y_test)}")

```

Train: 8000, Test: 2000

```

rf = RandomForestClassifier(n_estimators=300, max_depth=12, random_state=42)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)

print(classification_report(y_test, pred_rf, target_names=le.classes_))

feat_imp = pd.Series(rf.feature_importances_, index=FEATURES).sort_values(ascending=False)
print("Feature importances:\n", feat_imp)

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| LG           | 1.00      | 1.00   | 1.00     | 400     |
| LL           | 1.00      | 1.00   | 1.00     | 400     |
| LLG          | 1.00      | 1.00   | 1.00     | 400     |
| LLL          | 1.00      | 1.00   | 1.00     | 400     |
| No Fault     | 1.00      | 1.00   | 1.00     | 400     |
| accuracy     |           |        | 1.00     | 2000    |
| macro avg    | 1.00      | 1.00   | 1.00     | 2000    |
| weighted avg | 1.00      | 1.00   | 1.00     | 2000    |

Feature importances:

|             |          |
|-------------|----------|
| I2_mag      | 0.208238 |
| I0_mag      | 0.190062 |
| I_unbalance | 0.120867 |
| I1_mag      | 0.108367 |
| V_unbalance | 0.079543 |
| Vb          | 0.074675 |
| Vc          | 0.072051 |
| Va          | 0.063445 |
| Ia          | 0.029215 |
| Ib          | 0.027066 |
| Ic          | 0.026471 |

dtype: float64

```
svm = SVC(kernel="rbf", C=10, gamma="scale", probability=True, random_state=42)
svm.fit(X_train_s, y_train)
pred_svm = svm.predict(X_test_s)

print(classification_report(y_test, pred_svm, target_names=le.classes_))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| LG           | 1.00      | 1.00   | 1.00     | 400     |
| LL           | 1.00      | 1.00   | 1.00     | 400     |
| LLG          | 1.00      | 1.00   | 1.00     | 400     |
| LLL          | 1.00      | 1.00   | 1.00     | 400     |
| No Fault     | 1.00      | 1.00   | 1.00     | 400     |
| accuracy     |           |        | 1.00     | 2000    |
| macro avg    | 1.00      | 1.00   | 1.00     | 2000    |
| weighted avg | 1.00      | 1.00   | 1.00     | 2000    |

```
results = pd.DataFrame([
    {"Model": "Random Forest",
     "Accuracy": accuracy_score(y_test, pred_rf),
     "F1 (macro)": f1_score(y_test, pred_rf, average="macro")},
    {"Model": "SVM (RBF)",
     "Accuracy": accuracy_score(y_test, pred_svm),
     "F1 (macro)": f1_score(y_test, pred_svm, average="macro")},
])
print(results.to_string(index=False))
```

| Model         | Accuracy | F1 (macro) |
|---------------|----------|------------|
| Random Forest | 0.9995   | 0.9995     |
| SVM (RBF)     | 1.0000   | 1.0000     |

```
fig, axes = plt.subplots(2, 2, figsize=(13, 10))
```

```
feat_imp.plot(kind="barh", ax=axes[0, 0], color="#059669")
axes[0, 0].set_title("Random Forest Feature Importance")
axes[0, 0].invert_yaxis()
```

```
cm_rf = confusion_matrix(y_test, pred_rf)
axes[0, 1].imshow(cm_rf, cmap="Greens")
axes[0, 1].set_xticks(range(len(le.classes_))); axes[0, 1].set_xticklabels(le.classes_, rotation=45)
axes[0, 1].set_yticks(range(len(le.classes_))); axes[0, 1].set_yticklabels(le.classes_)
for i in range(len(le.classes_)):
    for j in range(len(le.classes_)):
        axes[0, 1].text(j, i, cm_rf[i, j], ha="center", va="center")
axes[0, 1].set_title("Confusion Matrix - Random Forest")
```

```
cm_svm = confusion_matrix(y_test, pred_svm)
axes[1, 0].imshow(cm_svm, cmap="Purples")
axes[1, 0].set_xticks(range(len(le.classes_))); axes[1, 0].set_xticklabels(le.classes_, rotation=45)
axes[1, 0].set_yticks(range(len(le.classes_))); axes[1, 0].set_yticklabels(le.classes_)
```

```

for i in range(len(le.classes_)):
    for j in range(len(le.classes_)):
        axes[1, 0].text(j, i, cm_svm[i, j], ha="center", va="center")
axes[1, 0].set_title("Confusion Matrix — SVM")

x_pos = np.arange(len(results))
axes[1, 1].bar(x_pos, results["Accuracy"], color=["#059669", "#7c3aed"])
axes[1, 1].set_xticks(x_pos); axes[1, 1].set_xticklabels(results["Model"])
axes[1, 1].set_ylim(0, 1.05)
axes[1, 1].set_title("Accuracy Comparison")

plt.tight_layout()
plt.show()

```



